

AGENT COLLABORATION & MULTI-AGENT ORCHESTRATION

Study Guide — How Specialized Agents Hand Off Work, Combine Results, and Stay Auditable Together

Week 5 · Day 2 · Prepared by Rajesh Pasham

Program Context

Item	Detail
Session Type	Live demonstration and architecture discussion — building directly on Day 1's single-agent foundations
Platform	Individual Palantir Developer Tier accounts, in continuous use since Week 1, Day 3
This Week's Lab (Day 3)	Multi-Agent Insurance Workflow — orchestrated agents that collaborate, invoke Actions and Functions, and write back to your Ontology
Skill Gained This Week	AI Agent Engineer — the fifth rung of the program's Executive Skills Matrix

Where Today Fits

Day 1 gave you everything one agent needs: planning, memory, tool invocation, and the guardrails that bound it. But Wednesday's lab doesn't build one agent — it builds two per batch, working together on a single goal. Today is about the mechanics that make that possible: how agents hand off work, how an Orchestrator decides who does what, and how the results of several agents combine into one trustworthy outcome.

This is not new theory in isolation — it is Day 1's architecture, multiplied. Every agent in today's patterns still plans, remembers, and invokes tools exactly as Day 1 described. What's new is the layer above a single agent: how its output becomes another agent's input, and who decides the order.

Learning Objectives

By the end of today, you will be able to:

- Explain why a single agent's scope should stay narrow, and what that implies for collaboration
- Describe agent-as-tool, sequential handoff, and supervisor-worker collaboration patterns
- Explain what an Orchestrator Agent actually does
- Identify the state, latency, and deadlock risks specific to multi-agent runs
- Describe how a multi-agent run stays auditable
- Describe your batch's own orchestration pattern for Wednesday's lab

Topic 1 — Agent Collaboration Fundamentals

1. Why a Single Agent Isn't Enough

- **A real goal often spans more than one responsibility** — “assess this claim end to end” touches coverage, fraud risk, and settlement, three responsibilities Day 1 deliberately split across three narrow agents.
- **One wide agent recreates the problem specialization solved** — folding claims, fraud, and settlement logic back into a single agent to avoid coordination brings back exactly the evaluation, security, and deployment costs Day 1's Topic 4 argued against.
- **Collaboration is the missing piece, not a new agent design** — every agent in today's patterns still plans, remembers, and calls tools exactly as Day 1 described; what's new is how one agent's output reaches another.

2. Three Shapes of Collaboration

Different goals call for different collaboration structures — Wednesday's lab uses more than one.

Pattern	How It Works	Best Suited To
Agent-as-tool	One agent is exposed to another simply as a callable tool, the same way a Function is	A narrow, well-defined sub-task — the Underwriting Agent calling the Risk Assessment Agent for a fresh score
Sequential handoff	Agent A completes its part, then explicitly hands the goal and its findings to Agent B	A clear stage boundary — Claims Agent finishing coverage validation before Fraud Agent begins
Supervisor-worker	An Orchestrator Agent decomposes the goal and assigns pieces to several worker agents, then combines results	Goals that need more than one specialist's input to reach a single combined decision

3. Agent-as-Tool: The Simplest Collaboration Pattern

- **An agent's goal-taking interface looks like a Function's input schema** — the calling agent doesn't need to know how the called agent reasons internally, only what goal to hand it and what shape of result to expect back.
- **This is exactly how the Underwriting Agent uses the Risk Assessment Agent** — rather than recomputing a risk score itself, the Underwriting Agent calls the Risk Assessment Agent as a tool and waits for a fresh score, the same way it would call any Week 3 Function.
- **Nesting is possible, but bounded** — an agent-as-tool can itself call further tools, but an unbounded chain of agents calling agents is exactly the runaway-scope risk Day 1's guardrails topic warned about.

4. Sequential Handoff, Concretely

One agent's completed findings become the next agent's starting context. In the reference pattern: the Claims Agent completes coverage validation, and hands off to the Fraud Detection Agent, which then investigates the flagged claim. The shared context passed between them is deliberately narrow — claim ID, coverage result, and claim amount — not the Claims Agent's full internal reasoning or unrelated memory.

5. Shared Context: What Actually Gets Passed Between Agents

- **The goal, and what's been established so far** — “investigate FNOL-2201 for fraud”, plus the coverage result already confirmed; the receiving agent doesn't repeat work the sending agent already did.
- **Only the relevant slice of short-term memory** — Day 1's short-term memory scoping still applies here; the Fraud Detection Agent gets the claim details it needs, not the Claims Agent's entire working memory for every claim it has ever touched.
- **A reference back to the originating agent and step** — so an audit trail can reconstruct the full chain later; this is what Topic 3's observability discussion depends on.

6. Conflict Resolution: When Two Agents Disagree

- **A real example: coverage says pay, fraud says hold** — the Claims Agent's coverage validation clears the claim for settlement, while the Fraud Detection Agent flags the same claim as a suspicious pattern match.
- **Disagreement is a signal to escalate, not a tie to break silently** — the Orchestrator does not pick whichever agent answered first, or average the two outcomes; a flagged conflict routes to the Compliance Agent or a human, per your batch's governance plan.
- **The conflict itself is worth recording** — that two specialized agents disagreed is useful information for Week 6's Evals; a pattern of frequent disagreement between two agents may mean one of their tools or prompts needs revision.

Topic 2 — The Orchestrator Pattern

1. What an Orchestrator Agent Actually Does

- **Decompose the goal** — break “assess this claim end to end” into the sub-goals each specialist agent actually handles: coverage validation, fraud review, settlement recommendation.
- **Route each sub-goal to the right agent** — decide which specialized agent is invoked for which piece, and in what order; sequential where one result gates the next, parallel where it doesn't.
- **Combine what comes back into one outcome** — synthesize the specialists' results into a single recommendation or writeback, resolving conflicts per Topic 1's rules rather than silently favoring one agent.

2. Routing Logic: Two Ways an Orchestrator Decides

Approach	How It Decides	Trade-off
Rules-based routing	A fixed condition (claim amount, product line, prior flags) determines which agent is invoked next	Fast, predictable, and easy to audit — but only handles the cases its author anticipated
LLM-based routing	The Orchestrator itself reasons over the goal and available agents to decide who to invoke	Handles novel or ambiguous cases — but is harder to predict and needs its own evaluation, per Week 6

3. Sequential vs. Parallel Execution

Execution Mode	How It Runs	When to Use It
Sequential	Each agent runs only after the previous one completes and hands off its result	When one agent's output is required input for the next — coverage before settlement
Parallel	Independent agents run at the same time, and the Orchestrator waits for all to complete before combining	When agents don't depend on each other's output — risk scoring and document classification on the same application

4. Combining Results: Aggregation and Synthesis

- **Structured aggregation** — each agent returns a defined output shape (Week 2's Output Struct pattern, extended); the Orchestrator merges these fields into one combined record rather than free-form text.
- **Synthesis for anything that isn't purely structured** — where agents disagree or provide qualitative reasoning, the Orchestrator (or a human, per the conflict-resolution rule) produces the final narrative recommendation.
- **The combined output is what gets proposed as an Action** — only after aggregation does the result reach Week 3's Ontology Actions; the writeback still represents one governed decision, not several competing ones.

5. The Orchestrator's Own Tools

- **A decomposition capability, not a fixed script** — the Orchestrator's own planning step (Day 1's Topic 2) is what breaks “assess this claim” into sub-goals; it is not a hardcoded if/then list of which agent runs when.
- **Specialist agents themselves are its tools** — where a Claims Agent's tools are Functions and Actions, the Orchestrator's tools are the Claims Agent, the Fraud Detection Agent, and the others; agent-as-tool, applied at the top level.
- **Its own memory tracks the run so far, not claim history** — the Orchestrator's short-term memory holds which sub-goals are done, which are pending, and what each specialist returned; exactly the state a supervisor needs to combine results correctly.

6. Walking a Full Claim Through Orchestration

FNOL-2201, from intake to a governed writeback — four agents, one Orchestrator:

1. Orchestrator receives the goal: “Assess FNOL-2201 end to end”

It decomposes this into three sub-goals: validate coverage, assess fraud risk, recommend settlement.

2. Sequential step: Claims Agent validates coverage first

Coverage is a gating fact — nothing downstream matters if the policy isn't active, so this step runs before anything else.

3. Parallel step: Fraud Detection Agent and a document check run together

Neither depends on the other's output, so the Orchestrator runs them concurrently to save time, per the parallel-execution guidance above.

4. Fraud Detection Agent flags a pattern; Claims Agent's draft recommendation said approve

This is the conflict scenario from Topic 1 — the Orchestrator does not resolve it silently.

5. Orchestrator routes the conflict to the Compliance Agent

Per your batch's governance plan, a coverage-vs-fraud conflict is exactly the case Compliance exists to adjudicate.

6. Combined result reaches a human for the final Action submission

The Orchestrator's synthesized recommendation, with the conflict and Compliance's read on it, becomes the case a human actually approves.

Topic 3 — Multi-Agent System Design

1. Supervisor vs. Decentralized Collaboration

Model	Who Decides	Governance Implication
Supervisor (Orchestrator-led)	One Orchestrator Agent holds routing and combination authority	A single point to secure, audit, and evaluate — the pattern this program builds toward
Decentralized (peer-to-peer)	Agents invoke each other directly, with no central coordinator	Harder to audit and secure — every agent-to-agent edge needs its own governance, which is why Wednesday's lab uses a supervisor model

2. State Management Across Agents

- **Shared state lives in the Ontology, not in an agent's private memory** — the Claim object itself, not a copy inside the Claims Agent's working memory, is the source of truth every collaborating agent reads from.
- **Each agent's own memory stays scoped to its own reasoning** — the Fraud Detection Agent's short-term memory holds its own investigation notes; it doesn't need, and shouldn't hold, the Claims Agent's internal reasoning steps.
- **A race condition is possible when two agents write concurrently** — if the Fraud Detection Agent and the Claims Agent could both propose an update to the same Claim object at once, the same Action-submission discipline from Week 3, one committed writeback at a time, prevents a corrupted result.

3. Latency and Cost of Multi-Agent Runs

- **Sequential hops add up** — four agents each taking a few seconds to plan and act means a claim assessment that could take well over a minute end to end, a cost invisible when you tested one agent alone on Day 1.
- **Parallel execution is a genuine lever, not just an optimization** — the parallel step in the walkthrough above exists specifically because fraud review and document classification don't depend on each other; running them concurrently is a real latency reduction, not a nice-to-have.
- **Cost scales with the number of agent invocations, not just tool calls** — each agent's own reasoning is itself an LLM call; a supervisor-worker pattern with five agents costs meaningfully more per run than a single agent, a trade-off Week 6's cost-management topic returns to.

4. Deadlock and Loop Risks

- **A cycle is a design defect, not a runtime fluke** — if the Fraud Detection Agent's output can trigger the Claims Agent, and the Claims Agent's output can trigger the Fraud Detection Agent again, a legitimate disagreement can loop indefinitely.
- **Bounded hop counts prevent runaway chains** — the same run-length guardrail from Day 1 applies at the multi-agent level; an Orchestrator should cap how many handoffs a single goal can trigger before forcing escalation.
- **Trace every trigger path before Wednesday, not after** — before wiring your batch's two agents together in Agent Studio, map out every condition under which each agent could invoke the other, and check for a cycle on paper first.

5. Designing for Observability Across Agents

- **Every hop carries its originating context forward** — the reference back to the sending agent and step, introduced in Topic 1's shared-context discussion, is what lets a single run be reconstructed end to end.
- **A trace should read like the scenario walkthrough above** — goal received, decomposed, routed, one conflict, one resolution, one final writeback; an auditor should be able to follow exactly that sequence from the logs alone.
- **This is Week 6's Evals and observability, arriving early** — multi-agent systems are precisely where LLMOps discipline stops being optional; Week 6 formalizes the tracing this topic has been assuming you'll want.

Topic 4 — Trust, Quality & Your Batch's Orchestration

1. Evaluating Multi-Agent Outcomes

- **Evaluate the combined outcome, not just each agent alone** — a Claims Agent and a Fraud Detection Agent can each individually score well on their own narrow task, while the combined orchestration still reaches the wrong final recommendation.
- **Track how often agents disagree, and how conflicts resolve** — a rising disagreement rate between two specific agents is itself a signal worth investigating; it may point to a stale tool, a bad prompt, or a genuine edge case neither agent handles well.
- **This becomes the Weekly Evaluation's "Agent Review"** — your program's Weekly Evaluation Framework specifically calls out Agent Review for this week; this is what that review actually looks at.

2. Auditability: Which Agent Did What

- **Every Action submission traces to a decision chain, not just a final result** — the Object Explorer submission from Week 3 now carries a reference to the full orchestration: which agents ran, in what order, and what each contributed.
- **A human reviewing an outcome should see the same walkthrough this session gave you** — goal, decomposition, routing, any conflict, and resolution, presented clearly enough that a non-technical reviewer can follow it.
- **This is not optional for regulated writebacks** — an insurance settlement decision made by a multi-agent system needs the same auditability a human decision-maker's reasoning would be expected to have.

3. Citation Across Agents

- **Each agent's contribution carries its own source** — the Fraud Detection Agent's flag cites the specific prior claims or pattern it matched against, the same way Week 4's document intelligence cited a specific chunk.
- **The combined outcome cites each contributing agent, not just the Orchestrator** — a claims handler reviewing the final recommendation should be able to see which specific agent raised which specific concern, not just a single opaque "AI recommendation."
- **This is what makes escalation useful rather than just noisy** — a human resolving the walkthrough's conflict needs to see exactly what the Claims Agent and the Fraud Detection Agent each based their conclusion on.

4. Why This Matters for Day 3's Lab

- **You will configure a supervisor pattern, not a decentralized one** — your batch's two agents connect through an Orchestrator, following Topic 3's governance argument for why this program uses that model.
- **You will decide sequential vs. parallel for your own two agents** — based on whether your second agent's work depends on your first agent's output, exactly as the routing-logic discussion above framed the choice.
- **You will define what happens on disagreement, before you need it** — deciding your batch's conflict-resolution rule on paper first avoids discovering the gap live, during the lab.

5. Your Batch's Orchestration Pattern for Day 3

Batch	Leading Agent → Second Agent	Handoff Trigger	Execution Mode
Batch 1 — India	Claims Agent → Fraud Detection Agent	Claim amount or pattern crosses your batch's threshold	Sequential
Batch 2 — India	Underwriting Agent → Risk Assessment Agent	A decision requires a current, not stale, risk score	Sequential
Batch 3 — Europe	Fraud Detection Agent → Compliance Agent	A flagged claim requires regulatory sign-off before referral	Sequential
Batch 4 — Europe	Customer Support Agent → Policy Recommendation Agent	A query exceeds support scope and needs a coverage or renewal recommendation	Sequential

6. Pre-Flight Checklist Before Wednesday's Lab

- **Which of your batch's two agents leads?** Confirm it matches how your use case actually flows, not just the default assignment above.
- **What triggers the handoff between them?** A threshold, a status change, or a query type — write the exact condition down, the same precision Week 1's Marking rules demanded.
- **Sequential or parallel?** Only parallel if the second agent's work never depends on the first agent's output — when in doubt, start sequential.
- **What happens on disagreement?** Escalate to a third agent, or to a human — decide this before your first test run surfaces a real conflict.
- **Where does the trail live?** Confirm every hop between your two agents carries enough shared context back to reconstruct the run afterward.

7. Common Pitfalls

- **Letting every agent talk to every other agent** — unrestricted agent-to-agent access is the multi-agent equivalent of skipping RBAC; route collaboration through a defined Orchestrator or handoff contract, not an open mesh.
- **No timeout on a handoff** — an agent waiting indefinitely for another agent's result can stall an entire run; Wednesday's lab requires every handoff to have a bounded wait.
- **Two agents that can trigger each other in a cycle** — a Fraud Agent that always re-triggers the Claims Agent, which re-triggers the Fraud Agent, is a deadlock waiting to happen; trace every trigger path before Wednesday for cycles.
- **Combining results without resolving disagreement** — if the Claims Agent recommends approval and the Fraud Agent recommends a hold, silently picking one is worse than surfacing the conflict to a human.
- **Treating the Orchestrator as exempt from governance** — an Orchestrator Agent still needs RBAC/ABAC on what it can route to and what it can see; orchestration is not a loophole around Week 1's access model.
- **Losing the audit trail across a handoff** — if Agent A's reasoning isn't visible once Agent B takes over, no one can reconstruct why the combined decision was made; every hop needs to stay in the log.

8. Glossary

Term	Meaning
Agent-as-Tool	One agent exposed to another as a callable tool, with a goal-taking interface like a Function's input schema
Sequential Handoff	One agent completing its part and explicitly passing the goal and findings to the next agent
Supervisor-Worker (Orchestration)	An Orchestrator Agent decomposing a goal, routing pieces to worker agents, and combining their results
Shared Context	The deliberately scoped goal, findings, and reference passed from one agent to another at a handoff
Rules-Based Routing	An Orchestrator's routing decision made by a fixed condition rather than the Orchestrator's own reasoning
LLM-Based Routing	An Orchestrator reasoning over the goal and available agents to decide which to invoke
Race Condition	Two agents attempting to write conflicting updates to the same shared state at once
Deadlock (Multi-Agent)	A cycle of agents triggering each other that never reaches a stopping condition
Agent Review	The Weekly Evaluation Framework's review method for scoring multi-agent collaboration and outcomes

9. Further Practice

Continue into Day 3, the hands-on Multi-Agent Insurance Workflow lab, where your batch configures its two agents in Agent Studio, wires them to existing Functions and Actions, defines the handoff trigger and execution mode decided above, and tests the orchestration end to end.